AN AI ENGINEER'S FIELD GUIDE

Beyond "Write Me SQL"

Building an evidence-driven AI workflow for data warehousing

Real development lessons. Clear agent responsibilities.
Safer context sharing. Tests with an honest boundary.

01 CONTEXT
What must be true?

02 RESPONSIBILITY
Who owns the change?

03 EVIDENCE
What did we prove?

By Derin Ege Evren

AI & Machine Learning Engineer

For data practitioners moving from isolated prompts to a repeatable way of working.

SEPTEMBER 2026 / ANONYMIZED EXAMPLES

01 / THE TURNING POINT

A successful run is not the same as a correct result.

The useful question changed from "Can AI write this?" to "How will I know it is right?"

I approach warehouse work with an AI and machine learning engineering background. During one development task, I wanted a final check before calling it finished. The procedures existed and the SQL looked reasonable, but some development tables were empty. A clean result said little about what would happen with real records.

That became a recurring lesson: AI can help produce a convincing answer before we have supplied enough evidence to justify it. My job was not to reject the assistance. It was to make the missing evidence visible.

A useful check	What it does not establish
The SQL parses.	The object names, mappings and runtime behavior are correct.
The procedure executes.	It processed representative data or preserved business meaning.
Row counts match.	The same records and values were produced.
Synthetic tests pass.	Source integration and production readiness are complete.

Why this matters in a warehouse

A warehouse can be wrong without crashing. A join can multiply amounts. An ID can be mistaken for a business code. A history load can overwrite meaning while preserving the number of rows. These failures are harder to notice than a syntax error.

THE SHIFT

I stopped treating generated SQL as the deliverable. The deliverable became a scoped change, an explanation of its assumptions, and evidence about its behavior.

This guide follows that shift: define the contract, keep durable task memory, separate agent responsibilities, protect the context, and test the actual procedures. You can start with one agent; the process matters more than the headcount.

02 / BETTER INPUTS

Give AI a contract, not just a request.

A small amount of precise context is more useful than a large amount of unexplained SQL.

TOO VAGUE

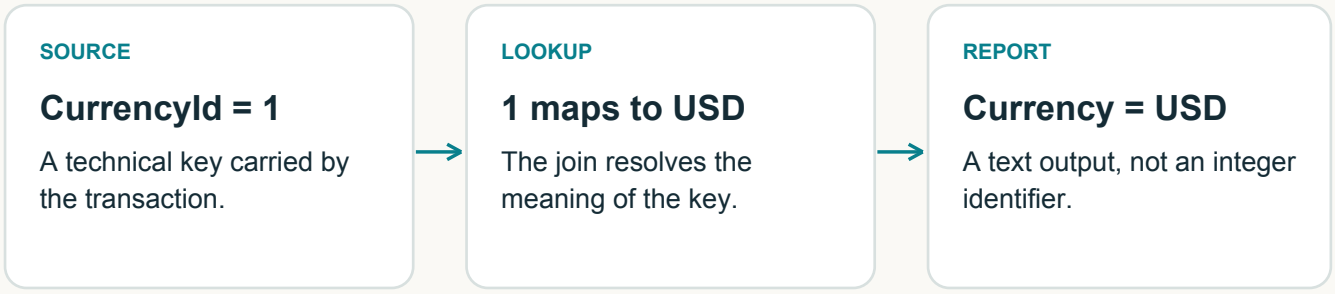
"Fix the joins and make this procedure correct."

A BOUNDED REQUEST

"Review the join keys against this mapping. Change only the approved join conditions. Preserve filters and output semantics. If an ID is missing upstream, flag it before editing. Return the minimal diff and an independent test."

My working contract includes five things: the goal, the sanitized source schema, the target mapping, the boundaries of the change, and the expected proof. If a business rule is unclear, I want a question rather than a plausible invention.

A concrete example: a key is not a display value



In one task, a currency code such as USD reached an integer output column. Checking the source metadata made the mismatch explicit. Renaming an NK (business-key) column to ID would not have solved it: a label does not change what the stored value means.

ILLUSTRATIVE EXAMPLE - NOT A DEPLOYMENT SCRIPT

```
-- Toy names; verify your own schema first.
LEFT JOIN CurrencyLookup AS c
    ON c.CurrencyId = t.CurrencyId
-- Report output: c.CurrencyCode, a text value
```

For a junior engineer, this is a useful habit: ask the assistant to explain the meaning on both sides of a join before asking it to rewrite the condition.

03 / PERSISTENT MEMORY

The conversation is not the system of record.

The framework began as a practical answer to lost context, repeated decisions and uncertain script versions.

Long chats made it difficult to answer simple questions: What is current? Why did we choose this mapping? Which script did I actually run? I moved those answers into a small task workspace. The assistant uses the files as working memory; I receive a short summary and one next action.

File or folder	Plain-English purpose
STATE.md	Where we are now; what is pending; the next human action.
DECISIONS.md	Why a choice was made, including the evidence and its scope.
LOG.md	The chronological record. New events do not erase old ones.
HANDOFF.md	A short restart note for another session or another agent.
evidence/	Results and their limits: observed, checked, or still unknown.
scripts/ + identifiers	Which revision was requested, executed, and accepted.

Keep claims at the right level

"Procedure created" is not the same state as "procedure tested." A screenshot may establish that an object exists without proving which complete revision ran. If I cannot tie an execution to the requested script, I record that uncertainty rather than silently assuming an exact match.

A USEFUL HANDOFF

Current: synthetic procedure tests passed.
Pending: source ingestion and historical start-date rule.
Do not rerun: older deployment revisions.
Next: inspect the staging package template.

This is file-based coordination, not a central autonomous control system. Persistent notes help a session restart without turning yesterday's assumption into today's fact.

04 / MODEL AND TOOL BREAKDOWN

Assign responsibilities before choosing tools.

This is a breakdown of our working arrangement, not a benchmark or a permanent ranking of models.

In the documented multi-agent workflow, Claude handled mapping and implementation ownership, Codex challenged assumptions and built tests, and Cursor hosted coordination and monitoring work. Those were assignments, not exclusive capabilities. In another session, Codex also owned implementation.

Tool / role in our setup	Typical output	Boundary
Claude **Implementation owner**	Mapping decisions, minimal SQL changes, state updates.	Do not overwrite another owner's work.
Codex **Reviewer / test author**	Counterexamples, parity checks, test scripts and review findings.	A favorable review is not runtime evidence.
Cursor workflow **Coordinator / narrator**	Shared-channel updates, change monitoring and a human summary.	Monitoring is not permission to rewrite SQL.
Human **Operator / decision owner**	Approved execution, returned results and operational decisions.	AI agreement does not replace approval.

IMPORTANT NAMING DETAIL

Claude is a model family; Codex is an agent/tool environment; Cursor is an editor/agent environment that can use different models. They are not three directly comparable model versions.

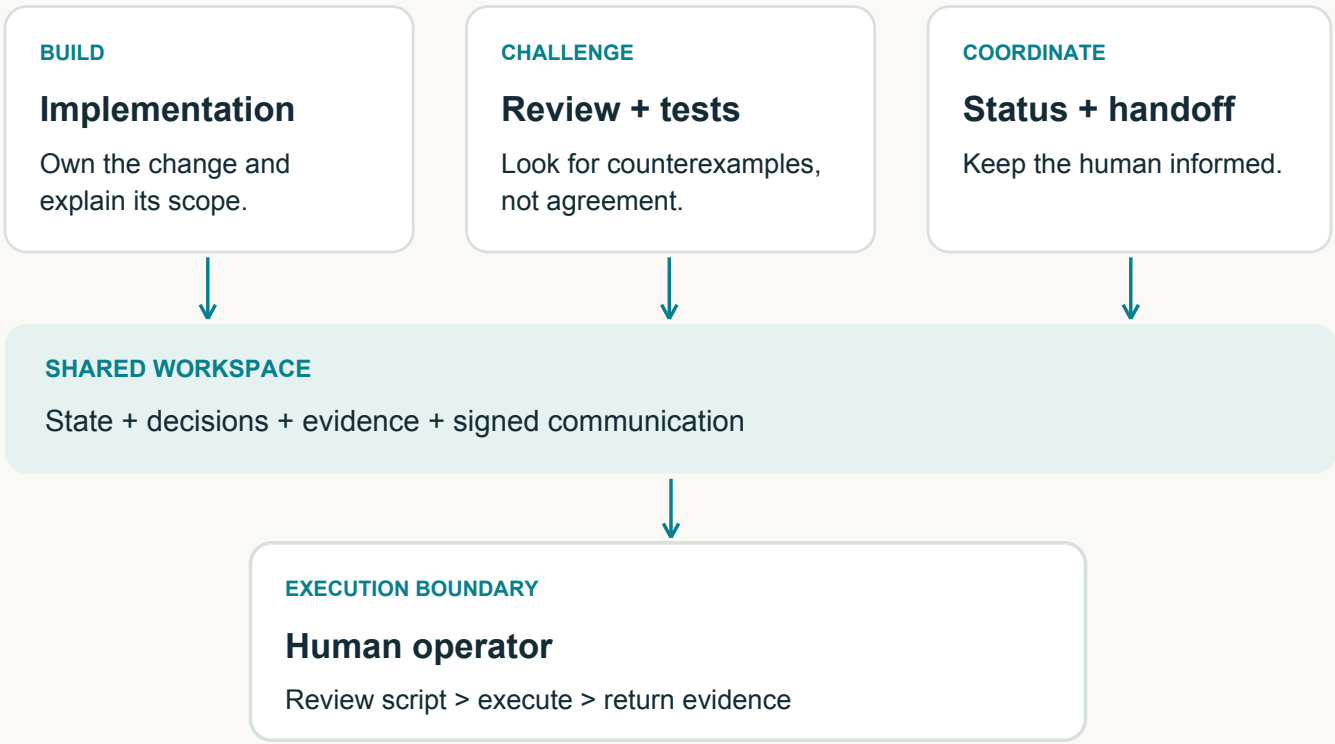
I would not publish a model-version leaderboard from these sessions: the exact versions were not consistently recorded, and there was no controlled performance or cost experiment. The useful lesson is the division of responsibility.

Choose an additional agent when it has a distinct job: investigate a join assumption, challenge a mapping, or design a test. Asking three agents to restate the same solution creates more text, not necessarily more assurance.

05 / ORCHESTRATION

Shared context. Clear ownership. One next action.

The agents communicate through a common file-based channel; the human remains the operator of corporate systems.



A shared channel keeps review requests and responses together. Each message has a sender and time. File ownership limits who edits what. Edit-intent messages announce cross-owner changes; script and evidence identifiers keep revisions traceable.

SIMPLIFIED EXAMPLE - NOT A VERBATIM CONVERSATION

```
BUILDER: Join change is ready for review.
REVIEWER: The output changed too. Out of scope.
BUILDER: Output restored; join-only diff attached.
COORDINATOR: Reviewed revision ready for testing.
```

The channel coordinates work; it does not enforce access. Real permissions and reliable write serialization are still needed to prevent unauthorized edits and collisions.

HUMAN-FRIENDLY ORCHESTRATION

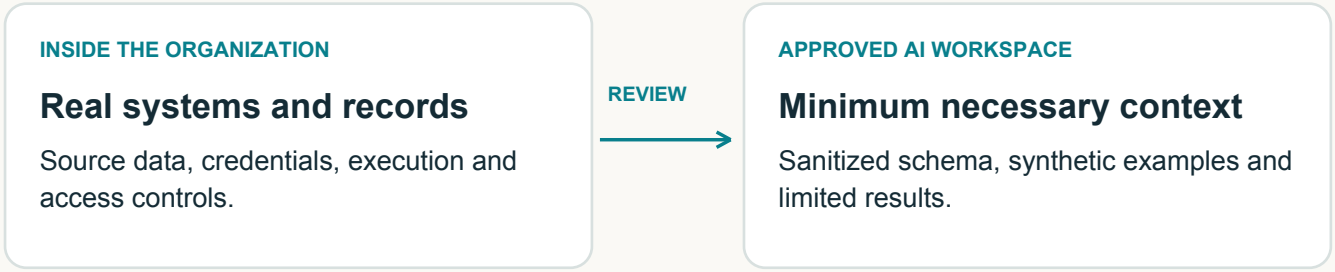
Agents work in parallel; the operator gets one clear next action. Resolve conflicting instructions before they reach the user.

06 / SECURITY

Share the structure, not the sensitive rows.

Useful AI context should be deliberately selected, not copied out of the environment by default.

My workflow keeps corporate execution with a human operator. That limits direct automated changes, but it does not make everything pasted into a chat safe. Data protection has a separate boundary: what leaves the environment, where it goes, who can access it, and how long it is retained.



A practical sharing checklist

Before sharing: remove credentials, customer identifiers, internal connection details and unnecessary business context. Review screenshots as carefully as text. Schema names and transformation rules can also be confidential.

For examples: use invented records that preserve the behavior you need to test. A NULL, an unmatched key and a boundary timestamp do not require a real customer. Simple masking may still leave a record identifiable.

For results: prefer needed counts, error categories and scoped PASS/FAIL outputs. Check error messages before forwarding them; they can contain actual values. Keep shared agent memory under the same scrutiny as the prompt.

For tooling: use the organization's approved service, account and data-handling settings. Verify access and retention requirements. A .gitignore rule only helps avoid accidental versioning; it does not control local access or AI-service retention.

NO BLANKET SECURITY CLAIM

These are risk-reduction practices, not proof that every historical exchange was sanitized or that a workflow is compliant. Instruction-level restrictions must not be mistaken for enforced technical controls.

07 / A REAL DEVELOPMENT CASE

Test the procedures, not just the draft.

A small synthetic dataset let us exercise the actual load path while real source ingestion was still pending.

The case involved four layers and three stored procedures. Staging held source-shaped records; a key table mapped warehouse IDs; a load layer prepared mapped rows; the warehouse stored history. Names are generalized here.

Layer	What we tested
STG / staging	Two synthetic source records, with GUIDs, timestamps and user references.
SKEY / key mapping	First load, stable mapping and no duplicate keys on repeat.
LRY / load layer	Column mapping, user lookup behavior and technical defaults.
EDW / history	Insert, change detection, closure and reactivation.

The important edge cases

One record had a missing creator and a deliberately unmatched modifier. The second used an existing, uniquely matched user key without exposing its value in the report. The shared user-key table was read, not modified.

We changed a GUID from NULL to the all-zero GUID: two different values. We changed a datetime2(7) value by 100 nanoseconds, then removed a source record and brought it back to test closure and reactivation.

ILLUSTRATIVE EXAMPLE - NOT A DEPLOYMENT SCRIPT

```
Before: 2020-01-01 10:20:30.1234567
After:  2020-01-01 10:20:30.1234568
Question: does a new history version appear?
```

A SCOPED COMPARISON TECHNIQUE

EXCEPT compared the business columns. It treats two NULLs as equal [1], but removes duplicates. Check duplicate multiplicity separately before claiming parity.

08 / EVIDENCE, NOT A VICTORY LAP

23 checks passed. Here is what that means.

The result supports the tested behavior. It does not make a universal claim about the pipeline.

23 / 23

CHECKS PASSED

No reported error. No skipped check.

Observed in this test	Still outside the test
Key mapping and repeat-load behavior.	Real source ingestion and full population coverage.
Load-layer values and technical defaults.	Every possible source anomaly or lookup ambiguity.
GUID and precise timestamp changes.	All same-day history-change scenarios.
Deactivation and same-key reactivation.	Initial historical CreatedDate loading policy.
No reversed interval in this fixture.	Production concurrency and performance at scale.
Cleanup after the test.	SSIS integration and operational readiness.

The test refused to start if the four target tables were not empty. The real procedures ran inside an outer test transaction; their definitions were not changed to add test logic. After ROLLBACK, all four row counts were zero and the session had no open transaction.

CLEANUP HAS A BOUNDARY

Rollback returned the test rows to their starting state. It does not promise that every side effect disappears: SQL Server identity values can still advance [2]. The test did not reset the identity counter.

A precise handoff: "Synthetic procedure tests passed. Source ingestion, initial history rules and SSIS remain to be validated." Progress, without implied completion.

09 / WHAT THE STRUCTURE ADDS

The payoff goes beyond writing SQL.

The useful gains come from a repeatable process, not simply from having more agents.

In this workflow, the added value was the package around the code: a recorded decision, a reviewer with a separate job, a test tied to a claim, and a handoff that stated what was still missing. These are qualitative benefits, not measured productivity gains.

01 / CONTINUITY

Less restarting from memory

State and handoff notes preserve the next step. A new session can resume without relying on the whole chat or asking the operator to retell every decision.

02 / REVIEW

A second line of questioning

A distinct reviewer can challenge join meaning, duplicate rows and history behavior while the implementation owner keeps the change focused.

03 / TRACEABILITY

An explainable result

Script identifiers, decisions and evidence make it easier to connect a result to a change - and to say when the executed revision is uncertain.

04 / LEARNING

Reasoning you can revisit

The record preserves why a key, type or test was chosen. For a junior engineer, this can become a learning aid rather than a collection of unexplained answers.

The extras: reusable assets and clearer handovers

A good test can become a reusable pattern. A clear task brief can become the next task's starting point. Explicit boundaries help another developer review the work without reconstructing the conversation. Reuse the pattern, not an old business rule: each new mapping still needs validation.

A CONCRETE GAIN FROM THIS CASE

The output was not only three procedures. It also included a test of repeat loads, precise changes and history behavior, plus a clear statement that source ingestion and SSIS were still pending.

Most of these benefits are available with one agent. Multi-agent coordination adds value when a genuinely separate investigation or review can run alongside implementation without creating conflicting edits.

10 / EXPERIENCE AND SPEED

Can AI help a junior catch up?

Separate the speed of producing an artifact from the judgment needed to trust it.

Some experienced engineers use no AI agents. That does not make their work outdated. They may carry years of knowledge about source behavior, incidents, business exceptions and operational risk. Age and tool adoption are not useful substitutes for assessing that expertise.

A junior can use this workflow to get help with drafting, understanding unfamiliar SQL and constructing tests. Those are opportunities to reduce friction, not proof of senior-level judgment. The learning comes from explaining the mapping, checking the assumptions and interpreting failures - not from accepting a generated answer.

COMPLEMENT, NOT REPLACEMENT

An experienced engineer can define the constraints and review the hard cases while a junior uses AI for bounded preparation and testing. A senior can also use the same tools. No job title implies a fixed AI advantage.

How much faster? Count the whole task.

Our 23-check result measured behavior, not speed. We cannot infer a percentage saving or a time-to-seniority from it. The example below uses invented minutes to show why faster drafting may or may not reduce total effort.

Illustrative scenario	Draft	Review	Test	Other*	Total
Without agents	40	30	30	0	100
AI-assisted, controlled	15	35	30	10	90
Same, plus rework	15	35	30	30	110

** Other = coordination and rework. Invented values, not a benchmark. Same scope and acceptance criteria are assumed.*

In this example, drafting is 62.5% shorter, but total effort is only 10% lower. Add 20 minutes of rework and total effort becomes 10% higher than the baseline. Compare workflows on equivalent tasks, not people on different problems.

Measure your own workflow: track hands-on time, elapsed delivery time, review corrections and later defects. Compare equivalent tasks at the same quality bar. Do not confuse elapsed time with summed human or agent effort.

11 / COSTS AND TRADE-OFFS

Useful structure is not free structure.

Every benefit has a maintenance cost. Keep the smallest workflow that addresses the risk.

Potential benefit	The downside	How to keep it useful
Durable memory	Stale notes can make yesterday's assumption look authoritative.	Update current state after evidence; record unresolved items explicitly.
More review	Agents can share a blind spot or create false confidence by agreeing.	Request counterexamples and independently check results.
Parallel work	Duplicate effort, editing conflicts, tool costs and waiting.	Give each agent a distinct output and a clear stopping point.
Traceability	Maintaining logs and identifiers can become paperwork.	Keep state short; archive history; record decisions that affect the work.
Human-controlled execution	The operator can become a bottleneck or copy the wrong revision.	Give one clear action and identify the exact script revision.
Shared context	More copies and participants widen the information-handling surface.	Minimize shared data and apply approved access and retention controls.

Choose the scale of the workflow

SMALL CHANGE

One agent

A bounded edit, known mapping and a focused test may be enough.

SEPARATE RISK

Add a reviewer

Use a second agent for a distinct mapping, parity or history check.

LONG-RUNNING WORK

Add coordination

Use persistent handoffs when sessions or multiple owners must stay aligned.

THE PRACTICAL DECISION

Keep the extra layer only if it improves review, continuity or clarity enough to justify its overhead. If the team spends more time coordinating than resolving the task, simplify it.

12 / LESSONS FROM FRICTION

More agents and more code are not always better.

The failures in the workflow were as useful as the successful SQL.

1. Helpful changes can exceed the task

I repeatedly had to narrow the request: change the join, not the filters; keep the temporary-table structure; follow the existing generator conventions. An assistant can improve a different problem from the one you authorized. Ask for a minimal diff and a list of everything intentionally left unchanged.

2. Two agents can agree on the same mistake

If both read an incomplete mapping, both can reach the same wrong conclusion. Give the reviewer a different job: find a counterexample, check the grain, or identify a condition that would invalidate the claim. Settle disagreements with evidence, not a model vote.

3. Bigger context can hide the important constraint

A huge procedure is not a substitute for a clear contract. Supply the relevant source definition, mapping and nearby logic. Keep the decision record short enough that a new session can identify what is current. Preserve old history without forcing every reader to consume all of it.

4. Coordination has an overhead

A second agent is useful when the subtask is distinct and the handoff is cheap enough. A one-line change may only need one agent and one focused test. Waiting, duplicated review and channel noise all count as work, even when no SQL is produced.

MEASURE BEFORE CLAIMING A SAVING

For future comparisons, record the task, exact model/version, tool, reasoning setting, review time, defects found and actual spend. This case has no controlled cost or speed benchmark, so it makes no percentage-improvement claim.

The objective is not maximum parallelism. It is a manageable process in which a reviewer can challenge the work and a human can understand the next decision.

13 / START SMALL

A starter workflow for a junior data engineer.

You do not need three tools or a complex agent framework to begin.

Step	Your practical action
01 / Bound	Choose one change. State the goal and what must stay unchanged.
02 / Sanitize	Share only approved, necessary context and a small synthetic example.
03 / Explain	Ask the assistant to describe the mapping and unknowns before editing.
04 / Change	Request the smallest reviewable diff, not a fresh rewrite by default.
05 / Challenge	Ask what would make the change wrong. Add a distinct reviewer if useful.
06 / Prove	Define expected results first, then run the approved test in the appropriate environment.
07 / Record	Save the revision, observed result, remaining limits and one next action.

A reusable task brief

COPY AND ADAPT

Goal: [one concrete change]
Inputs: [approved schema and mapping]
Preserve: [filters, outputs, conventions]
Unknowns: [items requiring confirmation]
Deliver: [minimal diff and separate test]
Prove: [expected rows, values and history behavior]
Authority: [what may be edited; who executes]

Ask questions that build your judgment: "What does this key represent?" "Which assumption are we making?" "Could these counts match while the data differs?" "What is the smallest test that could disprove this?"

14 / CLOSING NOTES

The real improvement was a better verification process.

AI helped with the work. Structure made the work easier to question, resume and verify.

What changed most was not how quickly a SELECT appeared. It was how I organized the uncertainty around it: a source contract instead of guesswork, explicit ownership instead of competing edits, and evidence instead of reassurance.

The framework is deliberately modest. Agents work with shared files and bounded responsibilities. The human operates the corporate environment. Another session can resume from a short handoff. A passed test is recorded with its scope, not promoted into a claim that everything is finished.

THE TAKEAWAY

The goal is not to stop thinking because AI can write SQL. It is to spend more of your thinking on meaning, risk and proof.

How to read the evidence in this guide

The workflow description comes from the practitioner's local framework documentation. The case result comes from a human-supplied SQL test report: 23 checks passed, no reported error, four empty tables after rollback and zero open transactions. It is not an independent audit or a model benchmark.

Examples and diagrams are generalized for explanation. The agent dialogue is illustrative, not a quotation. No corporate screenshots, customer rows, internal server names or credentials are reproduced. This does not establish that every earlier interaction met an organization's data policy.

Technical references

[1] Microsoft Learn - EXCEPT and INTERSECT (Transact-SQL). Supports the NULL comparison note on page 8.

[2] Microsoft Learn - CREATE TABLE: IDENTITY property. Explains that consumed identity values are not reused after a failed or rolled-back transaction.

Reference notes checked September 2026. Framework role assignments describe a working arrangement, not guarantees about model capabilities, costs or data protection.

ABOUT THE AUTHOR

Derin Ege Evren

AI & Machine Learning Engineer working across intelligent products and data systems.

Derin Ege Evren has a background in electrical and electronics engineering, with a focus on data analytics and artificial intelligence. His work spans machine learning, LLM-based automation, data engineering and end-to-end application development.

His project experience includes computer-vision-based fitness coaching with mobile applications and wearable integration; graph learning and reinforcement learning for scheduling; offline speech translation; and medical-image segmentation. His data work includes ETL development, database comparison and stored-procedure migration.

Why this perspective matters

This guide is not written only from the perspective of generating warehouse SQL. It approaches the development workflow as a system: context enters, agents have responsibilities, artifacts change, tests return evidence, and a human decides what happens next.

AI / ML

Build intelligence

Models, inference, automation and evaluation.

DATA SYSTEMS

Preserve meaning

Mappings, transformations and reliable data flows.

PRODUCT ENGINEERING

Connect the parts

Applications, interfaces and end-to-end behavior.

THE AUTHOR'S LENS

AI-assisted engineering should be understandable enough to question, bounded enough to control, and testable enough to improve. The framework is a working practice, not a claim to replace domain expertise.

Biographical details are based on the author-supplied CV. Contact details, personal address and date of birth are intentionally omitted. Results from other CV projects are not evidence of this framework's performance. The guide reflects the author's experience, not an employer's policy or endorsement.